Airport Connections

AlgoExpert · Very Hard · Graphs

The problem

- Minimum number of new one-way routes from the starting airport so every airport is reachable.
- Collapse strongly connected components (Tarjan) and count SCCs with in-degree 0, excluding the one containing the start.

Intuition

- Only unreachable airports matter; pick those that unlock the most others first.

Brute force vs optimal

- Greedy: repeatedly add route to largest unreachable set → O(V·(V+E))
- SCC condensation + count zero in-degree → O(V + E)

Approach

- 1. Find unreachable airports
- 2. For each, count airports reachable only through it
- 3. Sort by that count
- 4. Greedily connect

Worked example

```
airports + one-way routes; start LGA

goal: fewest new routes so every airport is reachable
```

Fewest new routes from the start

Worked example

```
collapse strongly connected components

→ a DAG of components
```

Collapse SCCs

Worked example

```
count components with no incoming edge

(excluding the start's component) → answer
```

Answer = sources other than start

Java solution (1/3)

```java
public class AirportConnections {
    public static int airportConnections(List<String> airports, List<List<String>> routes, String start) {
        Map<String, List<String>> g = new HashMap<>();
        for (String a : airports) g.put(a, new ArrayList<>());
        for (List<String> r : routes) g.get(r.get(0)).add(r.get(1));
        Map<String, Integer> comp = tarjan(airports, g);
        Set<Integer> hasIncoming = new HashSet<>();
        for (List<String> r : routes) {
            int a = comp.get(r.get(0)), b = comp.get(r.get(1));
            if (a != b) hasIncoming.add(b);
        }
        Set<Integer> all = new HashSet<>(comp.values());
        int count = 0;
        for (int c : all) if (!hasIncoming.contains(c) && c != comp.get(start)) count++;
        return count;
    }
```

Java solution (2/3)

```java
    private static Map<String, Integer> tarjan(List<String> nodes, Map<String, List<String>> g) {
        Map<String, Integer> idx = new HashMap<>(), low = new HashMap<>(), comp = new HashMap<>();
        Deque<String> st = new ArrayDeque<>();
        Set<String> on = new HashSet<>();
        int[] counter = {0, 0};
        for (String n : nodes) if (!idx.containsKey(n)) strong(n, g, idx, low, comp, st, on, counter);
        return comp;
    }
```

Java solution (3/3)

```
    private static void strong(String v, Map<String, List<String>> g, Map<String, Integer> idx, Map<String, Integer> low,
                               Map<String, Integer> comp, Deque<String> st, Set<String> on, int[] counter) {
        idx.put(v, counter[0]); low.put(v, counter[0]); counter[0]++;
        st.push(v); on.add(v);
        for (String w : g.get(v)) {
            if (!idx.containsKey(w)) { strong(w, g, idx, low, comp, st, on, counter); low.put(v, Math.min(low.get(v), low.get(w))); }
            else if (on.contains(w)) low.put(v, Math.min(low.get(v), idx.get(w)));
        }
        if (low.get(v).equals(idx.get(v))) {
            String w;
            do { w = st.pop(); on.remove(w); comp.put(w, counter[1]); } while (!w.equals(v));
            counter[1]++;
        }
    }
}
```

Complexity

- Time: O(a + r)
- Space: O(a + r)

Edge cases & pitfalls

- Exclude start's component
- Self-loops
- Already all reachable → 0

Interview tips & follow-ups

- Explain why sources suffice

Recap

- Only unreachable airports matter; pick those that unlock the most others first.
- Complexity: O(a + r)